

# ADVANCED EV3 PROGRAMMING LESSON



## PID Controller & Tuning for a Self-Balancing Robot

---

Ankit Ravankar  
30-05-2025

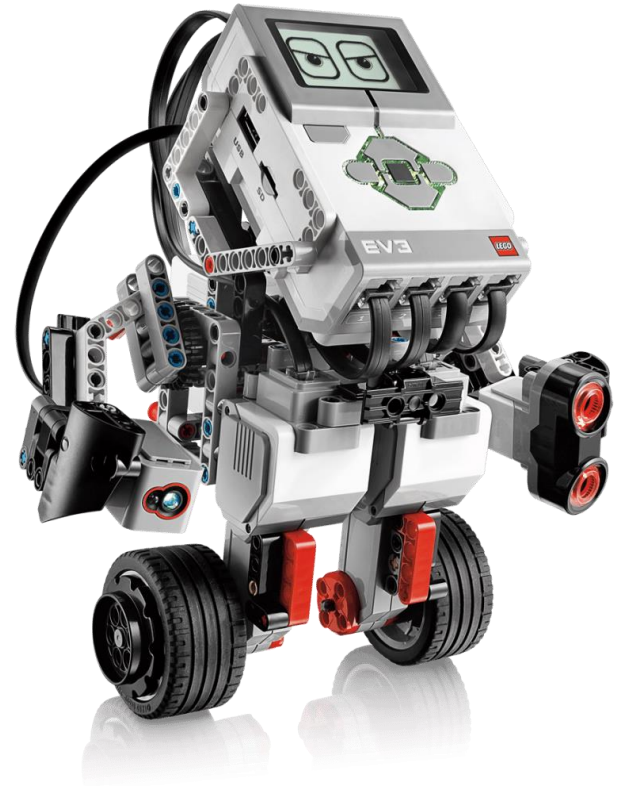
---

# Lesson Objectives

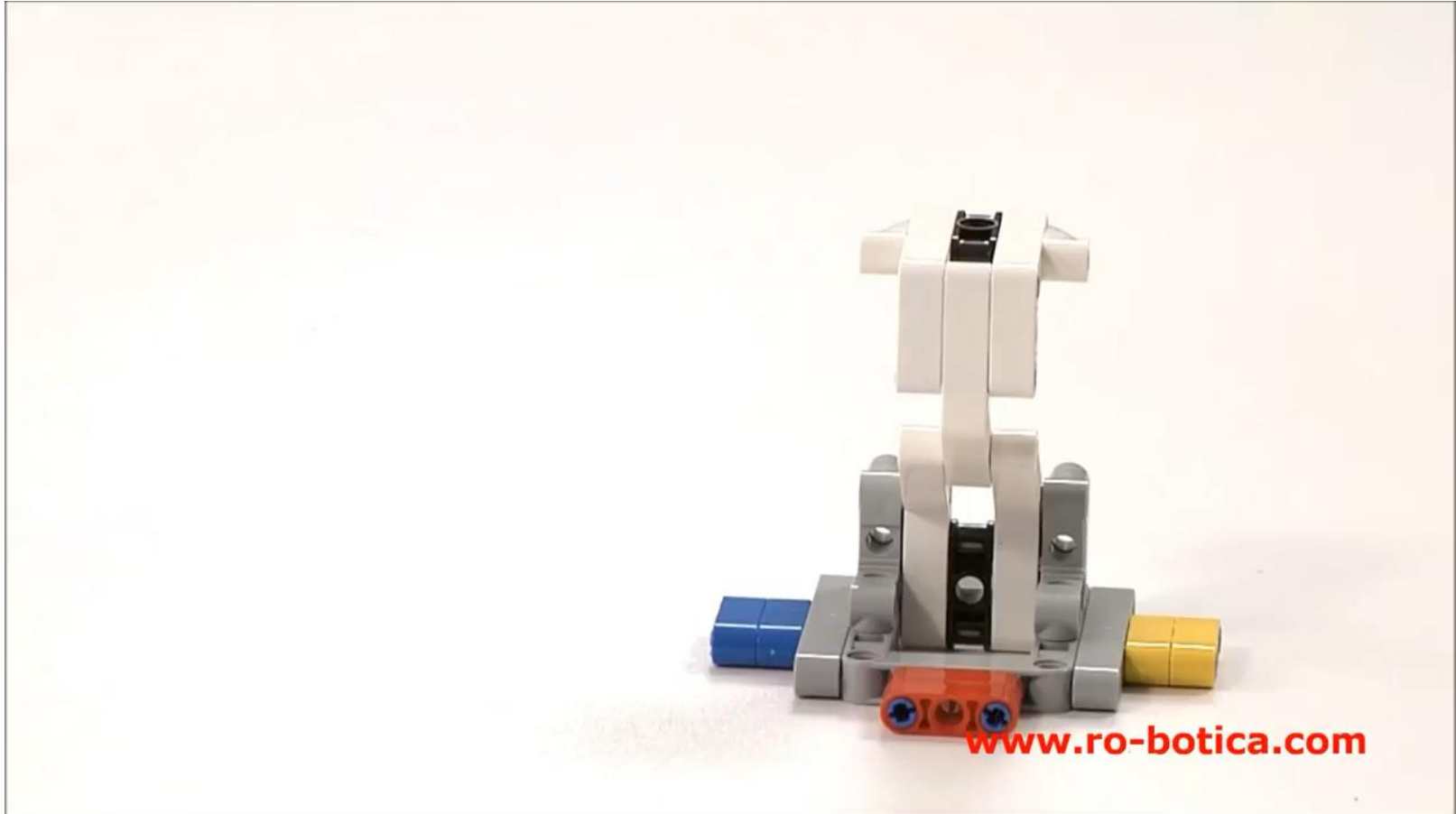
- Learn about PID controllers, how they work, and how to tune them
- Understand PID controller for inverse pendulum models (self-balancing robots)
- **Prerequisites:**
  - Basic programming in Python
  - VS Code installed on PC

# Gyroboy by LEGO Mindstorms

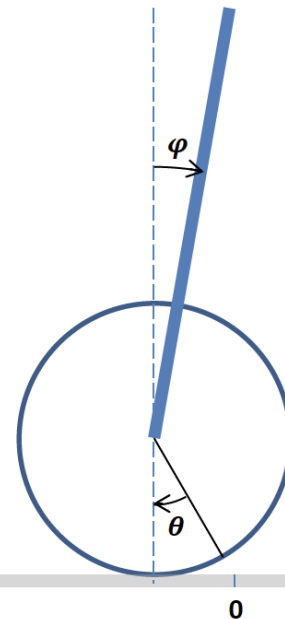
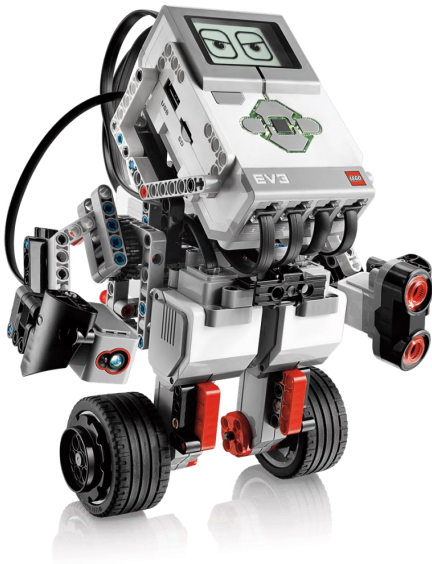
- Self balancing robot model from Mindstorm series.
- Demonstrates real-time control system and feedback.
- Inverse pendulum model on cart.
- Main sensors are
  - Two large motors with encoders
  - One gyro sensor
  - One ultrasound sensor
  - One color sensor
  - One touch sensor (Not used)



# Gyroboy in action



# Segway (Inverted pendulum)



# Gyroboy Model

$m$ : mass of the robot body

$M$ : mass of wheel base

$l$ : distance from wheel axle to center of gravity (CoG)

$I$ : moment of inertia of the body around its CoG

$g$ : gravitational constant

$F$ : horizontal force applied by the motors

## Kinetic Energy

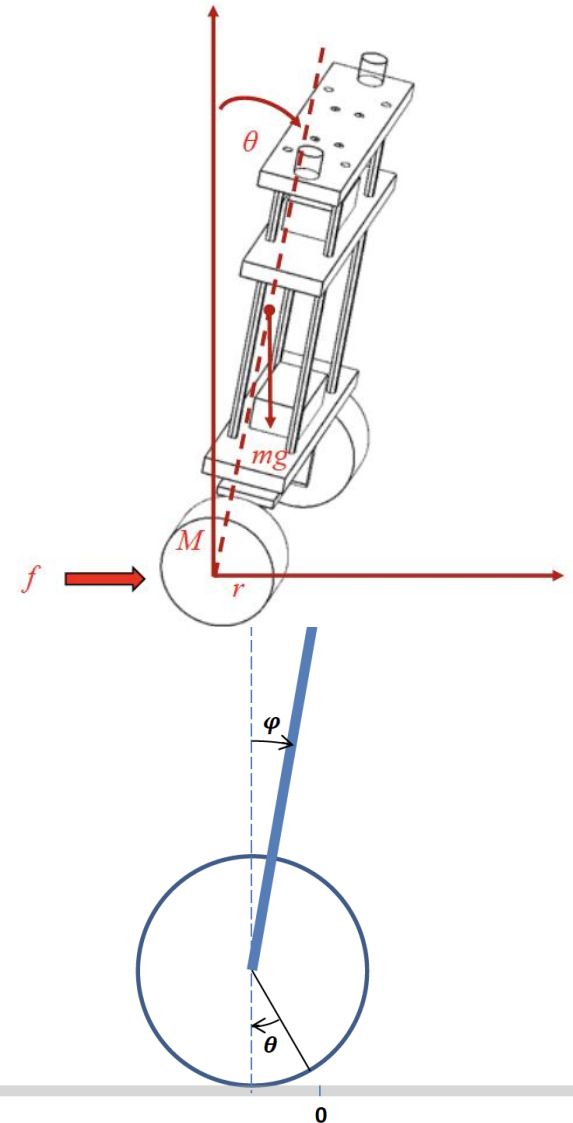
$$T = \frac{1}{2}M\dot{x}^2 + \frac{1}{2}m\left(\dot{x}^2 + 2\dot{x}l\dot{\theta}\cos(\theta) + l^2\dot{\theta}^2\right) + \frac{1}{2}I\dot{\theta}^2$$

## Potential Energy

$$V = -mgl\cos(\theta)$$

## Lagrangian

$$\mathcal{L} = T - V$$



# Inverse Pendulum model for gyroboy

Torque (Rotational)

$$I\ddot{\theta} + mgl \sin(\theta) = -ml\ddot{x} \cos(\theta)$$

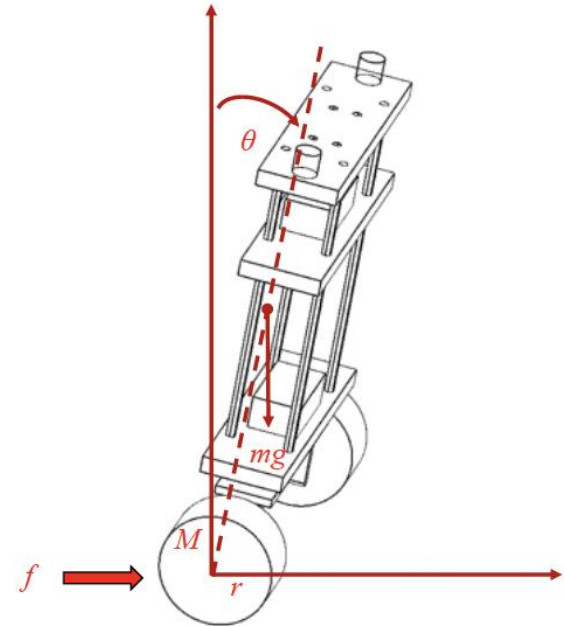
Force (Translational)

$$(M + m)\ddot{x} + ml\ddot{\theta} \cos(\theta) - ml\dot{\theta}^2 \sin(\theta) = F$$

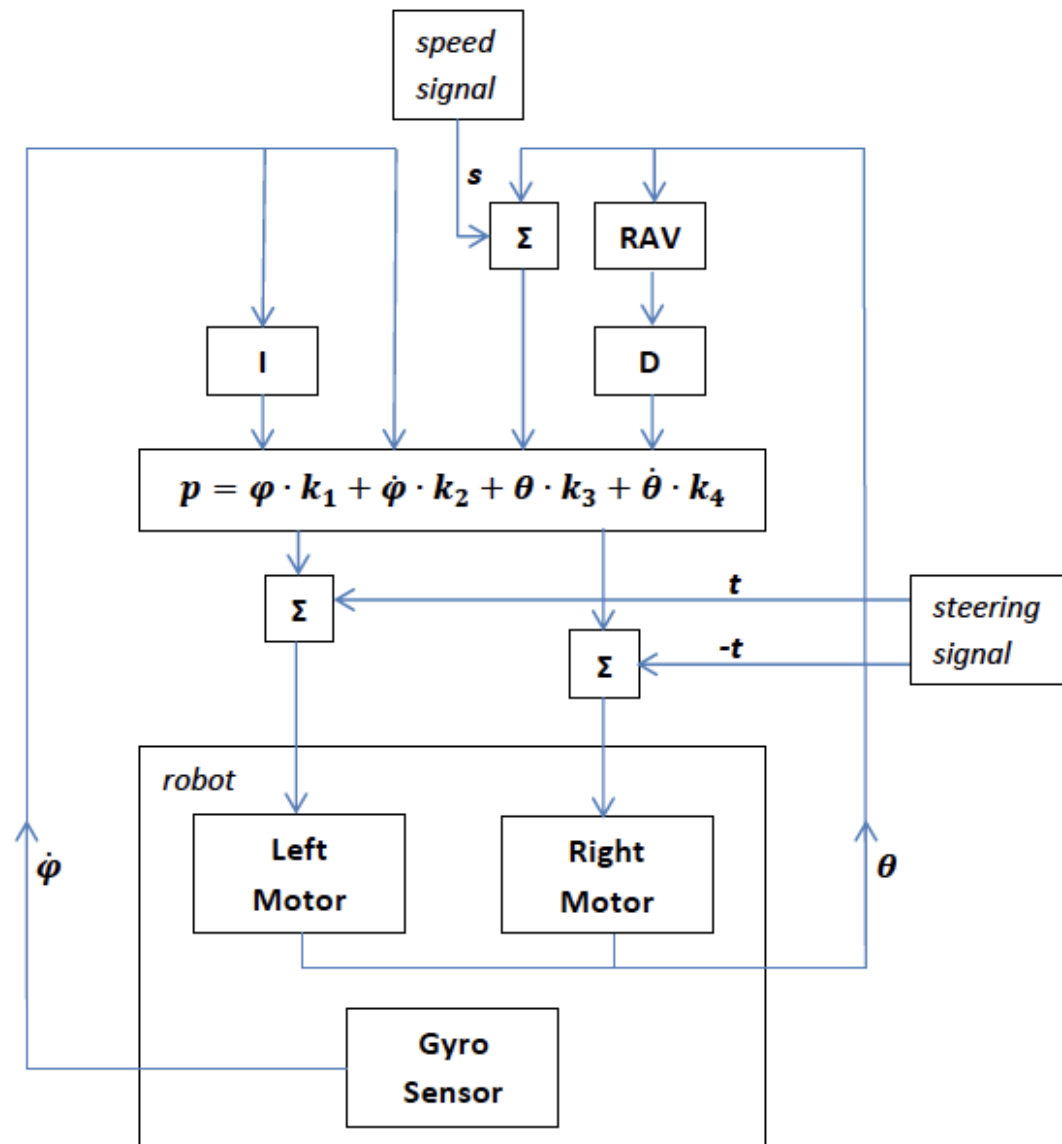
Simplified model for small theta

$$I\ddot{\theta} + mgl\theta = -ml\ddot{x}$$

$$(M + m)\ddot{x} + ml\ddot{\theta} = F$$

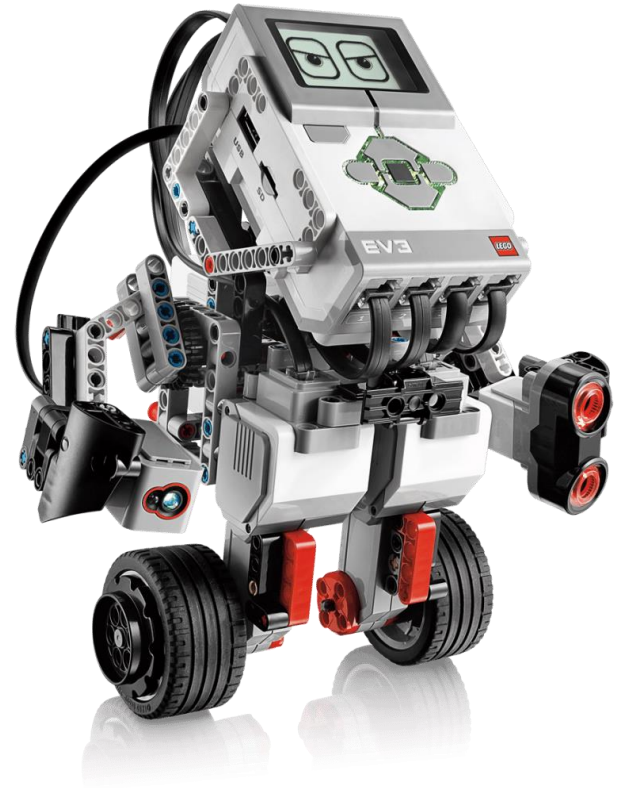


# Control Feedback Loop



# Gyroboy Platform

- We will use the gyroboy created using LEGO Mindstorms EV3
  - Programmable using LEGO building blocks, Scratch or MicroPython
  - Gyro sensor used for balancing.
- We will use VS Code to write and upload the MicroPython program



# Main component: Gyro sensor

- **The Gyro Sensor** detects **rotational** motion. If you rotate the Gyro Sensor in the direction of the arrows on the case of the sensor, the sensor can detect the rate of rotation in degrees per second. You can use the rotation rate to detect, for example, when a part of your robot is turning, or when your robot is falling over.
- In addition, the **Gyro Sensor** keeps track of the total rotation angle in degrees. You can use this rotation angle to detect, for example, how far your robot has turned.
- **Sensor Calibration:** The Angle and Rate can both be either positive or negative. Clockwise rotation is positive and counter-clockwise is negative.
  - • When connecting the Gyro Sensor to your EV3 Brick, you must hold it completely still in order to minimize “drifting”
  - • The Angle may “drift” over time and become less accurate.



Gyro sensor

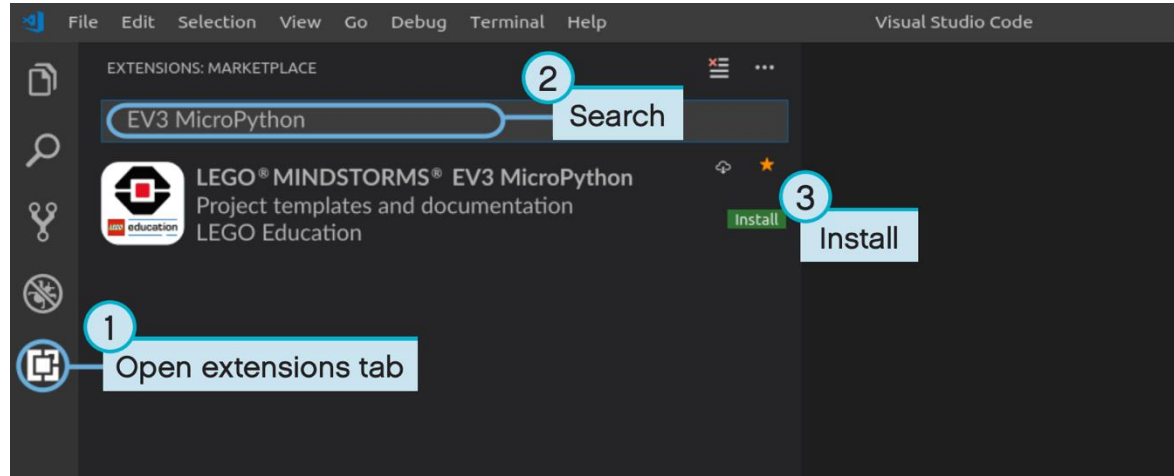
# Gyro Sensor Problems

- There are 2 common Gyro issues – **drift and lag**
  - Drift – readings keep changing even when the robot is still
  - Lag – readings are delayed
- Solution to drift: gyro calibration
  - The source of the drift problem is that the gyro must “learn” what is still.
  - For a color sensor, you have to “teach” the robot what is black and white
  - For your gyro, you need to calibrate the sensor to understand what is “still”

# Gyro Calibration to Solve Drift

- The gyro auto-calibrates when the robot is turned on or the gyro wire is connected. If the robot is moving during calibration, the gyro “learns” the wrong value for “still” – this causes drift!
- However, there is a way to force a calibration of the sensor → Stay still or start measuring only after few seconds you have left the robot alone!

# Environment Setup

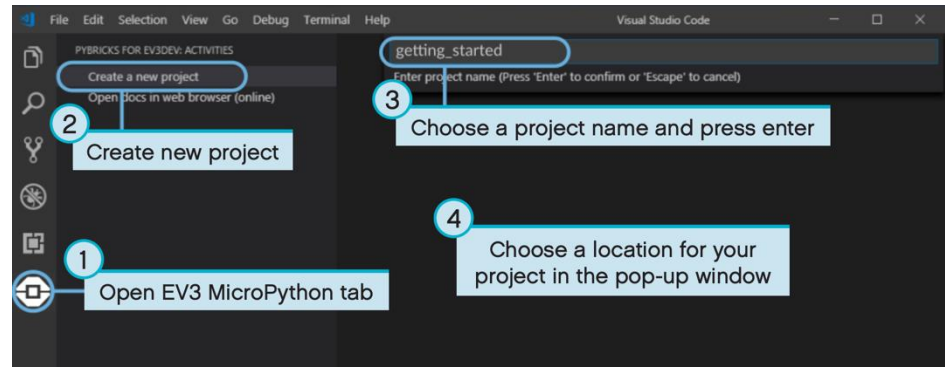


Same as last year

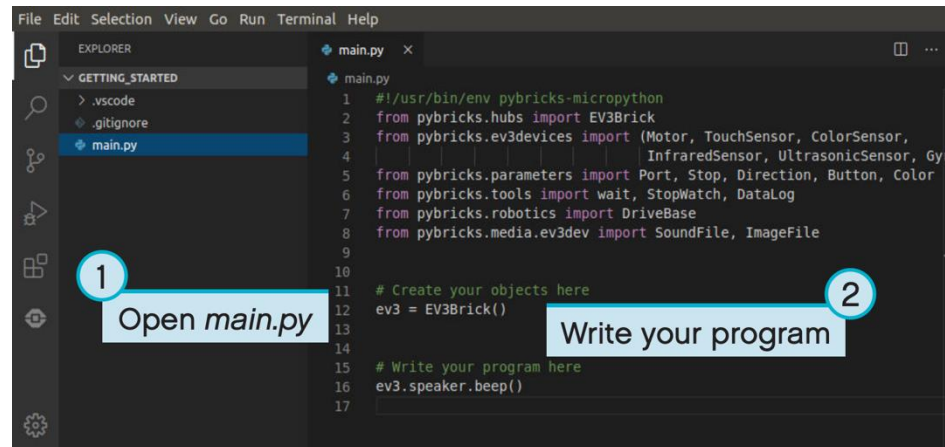
- Open VS Code
- Open the extensions tab
- Search for EV3 MicroPython
- Install the EV3 MicroPython extension

# Creating our EV3 Project

➤ Follow these steps to create a new project

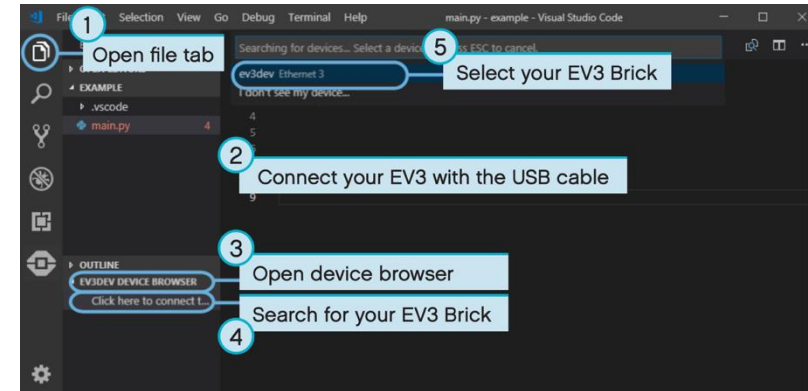


➤ After created, we'll be writing our code in the main.py file, that can be found in the file explorer.

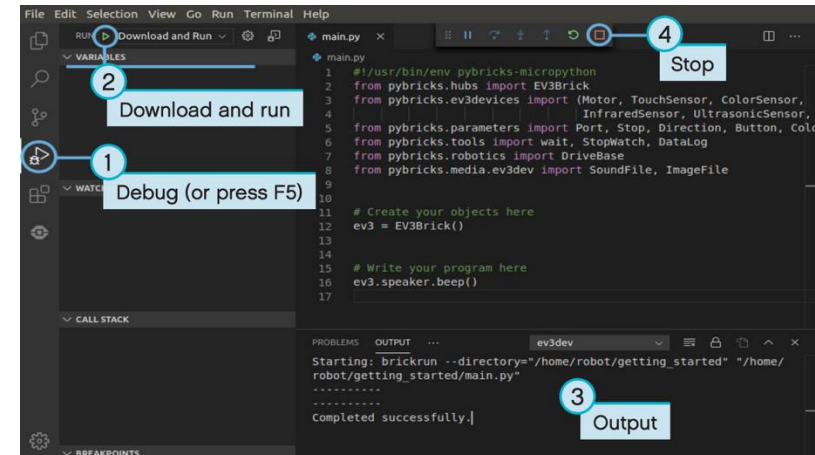


# Connecting and executing our program

➤ To connect the EV3, follow these steps →



➤ To run your program, follow this steps after connecting with your robot →



➤ Let's try executing the sample program!

# Understanding the gyroboy code

Sets the interpreter, then pulls in every Pybricks tool we need

PID gain block — for easy tuning

These lines bind Python variables to physical ports so later code can simply call `left_motor.dc(...)`, `gyro_sensor.speed()`, etc.

Take extra precaution here if the ports are different on your robot. Eg. For gyro sensor

```
7 This program requires LEGO® EV3 MicroPython v2.0.
8 Download: https://education.lego.com/en-us/support/mindstorms-ev3/python-for-ev3
9
10 Building instructions can be found at:
11 https://education.lego.com/en-us/support/mindstorms-ev3/building-instructions#building-core
12 """
13
14 from ucollections import namedtuple
15 import urandom
16
17 from pybricks.hubs import EV3Brick
18 from pybricks.ev3devices import Motor, UltrasonicSensor, ColorSensor, GyroSensor
19 from pybricks.parameters import Port, Color, ImageFile, SoundFile
20 from pybricks.tools import wait, StopWatch
21
22 #
23 # PID AND FEEDFORWARD GAINS (tune these only) -- Below are the best parameters
24 #
25 KF_DRIVE_SPEED = -0.01 # Feed-forward (bias) term
26 KD_BODY_RATE = 0.8 # Derivative term on body angular rate
27 KP_BODY_ANGLE = 15.0 # Proportional term on body angle
28 KD_WHEEL_RATE = 0.08 # Derivative term on wheel rate
29 KP_WHEEL_ANGLE = 0.12 # Integral-like term on wheel angle
30 #
31
32 # Initialize the EV3 brick.
33 ev3 = EV3Brick()
34
35 # Initialize the motors connected to the drive wheels.
36 left_motor = Motor(Port.D)
37 right_motor = Motor(Port.A)
38
39 # Initialize the motor connected to the arms.
40 arm_motor = Motor(Port.C)
41
42 # Initialize the Color Sensor. It is used to detect the colors that command
43 # which way the robot should move.
44 color_sensor = ColorSensor(Port.S1)
45
46 # Initialize the gyro sensor. It is used to provide feedback for balancing the
47 # robot.
48 gyro_sensor = GyroSensor(Port.S3)
49
50 # Initialize the ultrasonic sensor. It is used to detect when the robot gets
51 # too close to an obstruction.
52 ultrasonic_sensor = UltrasonicSensor(Port.S4)
53
```

# Understanding the gyroboy code

Separate timers for each action.

*Four independent StopWatches keep real-time control honest.*

Gyro calibration values. 15ms loop target, means our controller is high-speed.

Different action tuples, gives clear presets when interacting with colors.

*Maps sensor colours to high-level actions (speed + steering).*

```
54 # Initialize the timers.
55 fall_timer = Stopwatch()
56 single_loop_timer = Stopwatch()
57 control_loop_timer = Stopwatch()
58 action_timer = Stopwatch()
59
60
61 # The following (UPPERCASE names) are constants that control how the program
62 # behaves.
63
64 GYRO_CALIBRATION_LOOP_COUNT = 200
65 GYRO_OFFSET_FACTOR = 0.0005
66 TARGET_LOOP_PERIOD = 15 # ms
67 ARM_MOTOR_SPEED = 600 # deg/s
68
69 # Actions will be used to change which way the robot drives.
70 Action = namedtuple('Action', ['drive_speed', 'steering'])
71
72 # These are the pre-defined actions
73 STOP = Action(drive_speed=0, steering=0)
74 FORWARD_FAST = Action(drive_speed=150, steering=0)
75 FORWARD_SLOW = Action(drive_speed=40, steering=0)
76 BACKWARD_FAST = Action(drive_speed=-75, steering=0)
77 BACKWARD_SLOW = Action(drive_speed=-10, steering=0)
78 TURN_RIGHT = Action(drive_speed=0, steering=70)
79 TURN_LEFT = Action(drive_speed=0, steering=-70)
80
81 # The colors that the color sensor can detect are mapped to actions that the
82 # robot can perform.
83 ACTION_MAP = {
84     Color.RED: STOP,
85     Color.GREEN: FORWARD_FAST,
86     Color.BLUE: TURN_RIGHT,
87     Color.YELLOW: TURN_LEFT,
88     Color.WHITE: BACKWARD_FAST,
89 }
90
```

# Understanding the gyroboy code

## Behaviour generator: update\_action()

- **Yield** returns control to the balance loop without blocking.
- Generator + yield pattern delivers *non-blocking* behaviour updates.
- *Runs concurrently: reads colour/obstacle, yields new drive-speed or steering.*

```
119 # Start checking sensors on arms. When specific conditions are sensed,
120 # different actions will be performed.
121 while True:
122     # First, we check the color sensor. The detected color is looked up in
123     # the action map.
124     new_action = ACTION_MAP.get(color_sensor.color())
125
126     # If the color was found, beep for 0.1 seconds and then change the
127     # action depending on which color was detected.
128     if new_action is not None:
129         action_timer.reset()
130         ev3.speaker.beep(1000, -1)
131         while action_timer.time() < 100:
132             yield
133         ev3.speaker.beep(0, -1)
134
135     # If the new action involves steering, combine the new steering
136     # with the old drive speed. Otherwise, use the entire new action.
137     if new_action.steering != 0:
138         action = Action(drive_speed=action.drive_speed,
139                        steering=new_action.steering)
140     else:
141         action = new_action
142     yield action
143
144     # If the measured distance of the ultrasonic sensor is less than 250
145     # millimeters, then back up slowly.
146     if ultrasonic_sensor.distance() < 250:
147         # Back up slowly while wiggling the arms back and forth.
148         yield BACKWARD_SLOW
149
150         arm_motor.run_angle(ARM_MOTOR_SPEED, 30, wait=False)
151         while not arm_motor.control.done():
152             yield
153         arm_motor.run_angle(ARM_MOTOR_SPEED, -60, wait=False)
154         while not arm_motor.control.done():
155             yield
156         arm_motor.run_angle(ARM_MOTOR_SPEED, 30, wait=False)
157         while not arm_motor.control.done():
158             yield
159
160     # Randomly turn left or right for 4 seconds while still backing
161     # up slowly.
162     turn = urandom.choice([TURN_LEFT, TURN_RIGHT])
163     yield Action(drive_speed=BACKWARD_SLOW.drive_speed,
164                steering=turn.steering)
165     action_timer.reset()
166     while action_timer.time() < 4000:
167         yield
168
169     # Beep and then restore the previous action from before the
170     # ultrasonic sensor detected an obstruction.
171     action_timer.reset()
172     ev3.speaker.beep(1000, -1)
173     while action_timer.time() < 100:
174         yield
```

# Understanding the gyroboy code

Internal state variables

*These store  $\theta$ ,  $\phi$  and their derivatives between iterations.*

```
203
204     motor_position_sum = 0
205     wheel_angle = 0
206     motor_position_change = [0, 0, 0, 0]
207     drive_speed, steering = 0, 0
208     control_loop_count = 0
209     robot_body_angle = -0.25
210
```

Ensures robot is motionless; captures bias for drift correction.

```
# Calibrate the gyro offset. This makes sure that the robot is perfectly
# still by making sure that the measured rate does not fluctuate more than
# 2 deg/s. Gyro drift can cause the rate to be non-zero even when the robot
# is not moving, so we save that value for use later.
while True:
    gyro_minimum_rate, gyro_maximum_rate = 440, -440
    gyro_sum = 0
    for _ in range(GYRO_CALIBRATION_LOOP_COUNT):
        gyro_sensor_value = gyro_sensor.speed()
        gyro_sum += gyro_sensor_value
        if gyro_sensor_value > gyro_maximum_rate:
            gyro_maximum_rate = gyro_sensor_value
        if gyro_sensor_value < gyro_minimum_rate:
            gyro_minimum_rate = gyro_sensor_value
        wait(5)
    if gyro_maximum_rate - gyro_minimum_rate < 2:
        break
    gyro_offset = gyro_sum / GYRO_CALIBRATION_LOOP_COUNT
```

Balancing control loop (runs forever until fall)

```
# Main control loop for balancing the robot.
while True:
    # This timer measures how long a single loop takes. This will be used
    # to help keep the loop time consistent, even when different actions
    # are happening.
    single_loop_timer.reset()

    # This calculates the average control loop period. This is used in the
    # control feedback calculation instead of the single loop time to
    # filter out random fluctuations.
    if control_loop_count == 0:
        # The first time through the loop, we need to assign a value to
        # avoid dividing by zero later.
        average_control_loop_period = TARGET_LOOP_PERIOD / 1000
        control_loop_timer.reset()
    else:
        average_control_loop_period = (control_loop_timer.time() / 1000 /
                                         control_loop_count)
    control_loop_count += 1
```

# Understanding the gyroboy code

| Variable         | Calculation                                                   | Physical meaning |
|------------------|---------------------------------------------------------------|------------------|
| robot_body_rate  | gyro_sensor_value - gyro_offset                               | $\dot{\theta}$   |
| robot_body_angle | $\theta = \dot{\theta} \cdot dt$                              | $\theta$         |
| wheel_angle      | $\phi = \Delta \text{encoder} - \text{drive\_speed} \cdot dt$ | $\phi$           |
| wheel_rate       | mean of last 4 $\Delta \text{encoder} / dt$                   | $\dot{\phi}$     |

State estimation and meaning wrt variables

Control law

Feed-forward for robot leaning

P action on Body tilt

D action on Body tilt

PD like for wheel Motion, also consider as I term

$$u = k_f v_d + k_{\dot{\theta}} \dot{\theta} + k_{\theta} \theta + k_{\dot{\phi}} \dot{\phi} + k_{\phi} \phi$$

Motor actuation & steering

```
# calculate robot body angle and speed
gyro_sensor_value = gyro_sensor.speed()
gyro_offset *= (1 - GYRO_OFFSET_FACTOR)
gyro_offset += GYRO_OFFSET_FACTOR * gyro_sensor_value
robot_body_rate = gyro_sensor_value - gyro_offset
robot_body_angle += robot_body_rate * average_control_loop_period

# calculate wheel angle and speed
left_motor_angle = left_motor.angle()
right_motor_angle = right_motor.angle()
previous_motor_sum = motor_position_sum
motor_position_sum = left_motor_angle + right_motor_angle
change = motor_position_sum - previous_motor_sum
motor_position_change.insert(0, change)
del motor_position_change[-1]
wheel_angle += change - drive_speed * average_control_loop_period
wheel_rate = sum(motor_position_change) / 4 / average_control_loop_period
```

```
# This is the main control feedback calculation.
#output_power = (-0.01 * drive_speed) + (0.8 * robot_body_rate +
#15 * robot_body_angle +
#0.08 * wheel_rate +
#0.12 * wheel_angle)

output_power = (KF_DRIVE_SPEED * drive_speed
+ KD_BODY_RATE * robot_body_rate
+ KP_BODY_ANGLE * robot_body_angle
+ KD_WHEEL_RATE * wheel_rate
+ KP_WHEEL_ANGLE * wheel_angle)
#output_power = max(min(output_power, 100), -100)
if output_power > 100:
    output_power = 100
if output_power < -100:
    output_power = -100
```

```
# Drive the motors.
left_motor.dc(output_power - 0.1 * steering)
right_motor.dc(output_power + 0.1 * steering)
```

# Understanding the gyroboy code

## ➤ Fall detection

```
297  
298  
299 # Check if robot fell down. If the output speed is +/-100% for more  
300 # than one second, we know that we are no longer balancing properly.  
301 if abs(output_power) < 100:  
302     fall_timer.reset()  
303 elif fall_timer.time() > 1000:  
304     break
```

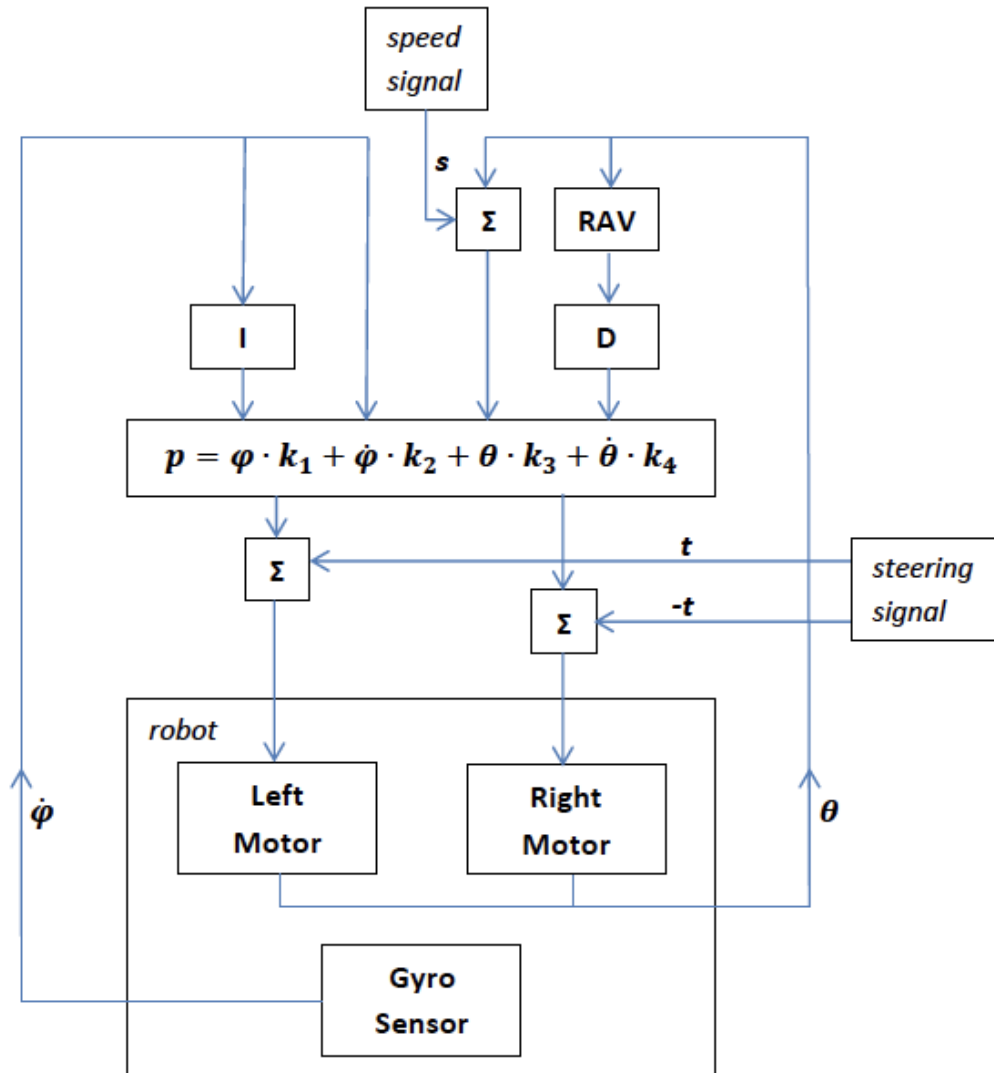
## ➤ Update driver command

```
305  
306 # This runs update_action() until the next "yield" statement.  
307 action = next(action_task)  
308 if action is not None:  
309     drive_speed, steering = action
```

## ➤ Fall recovery

```
315  
316 # Handle falling over. If we get to this point in the program, it means  
317 # that the robot fell over.  
318  
319 # Stop all of the motors.  
320 stop_action()  
321 left_motor.stop()  
322 right_motor.stop()  
323  
324 # Knocked out eyes and red light let us know that the robot lost its  
325 # balance.  
326 ev3.light.on(Color.RED)  
327 ev3.screen.load_image(ImageFile.KNOCKED_OUT)  
328 ev3.speaker.play_file(SoundFile.SPEED_DOWN)  
329  
330 # Wait for a few seconds before trying to balance again.  
331 wait(3000)
```

# Control Feedback Loop



Feed-forward for robot leaning

P action on Body tilt

D action on Body tilt

PD for wheel Motion, also consider as 'I' term

$$u = k_f v_d + k_{\dot{\theta}} \dot{\theta} + k_{\theta} \theta + k_{\dot{\phi}} \dot{\phi} + k_{\phi} \phi$$

Feed Forward Bias + Body PD + wheel PD

| Gain ↑                         | Expected behaviour       | If too high                |
|--------------------------------|--------------------------|----------------------------|
| $k_{\theta}$ (P)               | Corrects lean faster     | Oscillates, jitters        |
| $k_{\dot{\theta}}$ (D)         | Damps the wobble         | Sluggish, noisy            |
| $k_{\phi}$ ("I")               | Removes long drift       | Slow rocking or creep      |
| $k_{\dot{\phi}}$ (D on wheels) | Sharper speed changes    | Buzz / squeal              |
| $k_f$ (feed-forward)           | Rolls level at set speed | Constant lean if mis-tuned |

# What is PID?

- Proportional [Error] → How bad is the situation now?
- Integral → Have my past fixes helped fix things?
- Derivative → How is the situation changing?
- PID control → combine the error, integral and derivative values to decide how to steer the robot

# Key Step: Tuning The PID constants

➤ The most common way to tune your PID constants is **trial and error**.

| Task #                                    | Goal & Instructions                                                                                                                                                                                                         | Gain(s) to Touch                                  | Safe Start Range                                                                           | What You'll See                                                                                                                                                                                                                                                                                               | How to Recover                                                                                                                             |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1. Pure-P “Stand-Up”</b>               | Make GyroBoy stand $\geq 5$ s with <i>only</i> proportional feedback.1. Set all gains = 0 except KP_BODY_ANGLE.2. Start at low value, increment until stable.                                                               | KP_BODY_ANGLE                                     | 5 → 25                                                                                     | <ul style="list-style-type: none"> <li>► <b>Too low (&lt; 8)</b>: robot drifts then tips.</li> <li>► <b>Sweet spot (12–18)</b>: stands but oscillates <math>\pm 5^\circ</math>.</li> <li>► <b>Too high (&gt; 22)</b>: fast wobble → falls.</li> </ul>                                                         | <ul style="list-style-type: none"> <li>- Press center button to stop program.</li> <li>- Restore previous value; re-flash.</li> </ul>      |
| <b>2. Add D “Calm the Wiggle”</b>         | Keep the P you found in Task 1. Now add derivative on body rate to damp the oscillation. Increase slowly.                                                                                                                   | KD_BODY_RATE                                      | 0.2 → 1.2                                                                                  | <ul style="list-style-type: none"> <li>► <b>0</b>: large oscillations.</li> <li>► <b>0.5–0.9</b>: quick settle, minimal overshoot → ideal.</li> <li>► <b>&gt; 1.0</b>: sluggish response or jitter from gyro noise.</li> </ul>                                                                                | <ul style="list-style-type: none"> <li>- Lower the D value if motors buzz.</li> <li>- If still unstable, cut power, reboot.</li> </ul>     |
| <b>3. Rolling Balance &amp; Drift Fix</b> | Let the robot roll $\sim 1$ m forward and stop upright. 1) Keep gains from Task 2.2) Set KF_DRIVE_SPEED = -0.02 (feed-forward).3) Gradually raise KP_WHEEL_ANGLE and KD_WHEEL_RATE together to remove long-term tilt/drift. | KF_DRIVE_SPEED<br>KP_WHEEL_ANGLE<br>KD_WHEEL_RATE | KF_DRIVE_SPEED<br>-0.05 → -0.02<br>KP_WHEEL_ANGLE<br>0 → 0.25<br>KD_WHEEL_RATE<br>0 → 0.15 | <ul style="list-style-type: none"> <li>► <b>Low wheel gains</b>: robot leans while rolling, never levels out.</li> <li>► <b>Good mix</b>: leans briefly, accelerates, then straightens and coasts upright.</li> <li>► <b>Too high (&gt; 0.25/0.15)</b>: aggressive brake → overshoot, oscillation.</li> </ul> | <ul style="list-style-type: none"> <li>- Drop both wheel gains 50 %.</li> <li>- Reset to baseline if perpetual rocking appears.</li> </ul> |

# Bonus



- Demo 1: Make the robot go straight 1 m for 3s and then spot turn 360 deg.
- Demo 2: Make the robot run 1m x 1m square
- Hint: You can override some of the `update_action()`. Pick a color and do the task.

# Practical Demo



Demo with the treadmill control using PID.